AeroBench

Build spec for Claude Code. One native macOS app with four Formula Student aero tools, built crude first and polished later.

Dominic VerSoy · Coventry University Formula Student · 27 October 2026 · Working title, rename freely

0 How to use this document

This is a build brief for Claude Code. Read all of it before writing any code, then work through the phases in order. Each phase ends with acceptance checks, and a phase is not done until those pass.

Version 1 is deliberately crude. Default SwiftUI controls, plain layouts, no visual polish. A working pipeline with honest numbers matters far more than looks. If something here turns out impractical, pick the simplest approach that works, record it in `docs/DECISIONS.md` and keep going. Only stop to ask Dominic about the items in section 11 marked as his call.

This spec was written in October 2026. Package names, versions and install routes may have moved, so verify them at build time rather than trusting this document blindly.

1 What the app is

One app, four tools, sharing a project system, a Python compute engine and one 3D rendering core.

Tool	What it does	In v1
Wing Lab	Build multi-element wings from a draggable airfoil library, preview them instantly, run 2D RANS and optimise gaps, overlaps and angles. Shape optimisation later.	Library, canvas, inviscid preview, single SU2 run, gap and angle optimisation
CAD Viewer	Open and inspect STEP, IGES, STL and OBJ files quickly.	Load, orbit, standard views, part visibility, measure
Flow Field Viewer	Rebuild a 3D total pressure coefficient (Cp0) field from a front-to-back video of section planes and explore it.	Calibrate, invert colormap, stack volume, isosurfaces, slice scrubber
Lap Sim	Point-mass lap sim that turns aero changes into lap time.	Track import, car setup, lap time, speed trace, downforce vs drag exchange rate

Not in v1. Polished UI, 3D CFD, any STAR-CCM+ integration, shape optimisation, packaging for other machines, Windows support. Results are for fast iteration and ranking ideas, never final numbers.

2 Target machine

M5 Max MacBook Pro, 128 GB RAM, 2 TB, Apple silicon, latest macOS. Build for Apple silicon only. STAR-CCM+ runs separately inside a Parallels VM on the same machine and the app does not touch it in v1, but CFD jobs from the app should respect a configurable core limit so both can run at once.

3 Architecture

Front end

A SwiftUI app in an Xcode project. A `NavigationSplitView` sidebar lists Projects and the four tools. Swift Charts handles every 2D plot. The 3D core uses RealityKit behind a small protocol (call it `Viewer3D`) so the renderer can be swapped. If RealityKit fights custom meshes with per-vertex colour or fast mesh replacement, SceneKit is acceptable for v1, logged in DECISIONS.md.

Engine

A Python 3.12 package called `aeroengine` in `engine/`, managed with uv. All numerics live here. Every operation is a CLI subcommand that reads a JSON job file and writes results into a job folder.

```
aeroengine <command> --input job.json --workdir <job folder>

# stdout is JSON lines, one event per line
{"type": "progress", "pct": 42, "msg": "meshing"}
{"type": "log", "level": "info", "msg": "..."}
{"type": "result", "path": "result.json"}
{"type": "error", "msg": "gmsh failed", "detail": "..."}
```

Swift launches the engine with `Process`, streams the lines into the UI, and can cancel a job (SIGTERM, then SIGKILL after a short grace period). No local server. In v1 the app runs the engine from the repo's uv environment, with the path set in Settings. Bundling the engine inside the .app comes later.

Why the split. Swift gives a properly native Mac app. Python is where SU2 wrappers, gmsh, Optuna, OCP, OpenCV and scikit-image all live, and it keeps the maths testable on its own.

Repo layout

```
AeroBench/
  app/              Xcode project, SwiftUI
  engine/           aeroengine package, pyproject.toml, tests/
    scripts/        fetch_airfoils.py and other one-off tools
  data/airfoils/    airfoil library (fetched, see 8.1)
  samples/          small test inputs (STEP, video, tracks)
  docs/             this spec, DECISIONS.md
  README.md         setup steps that actually work on Dominic's Mac
```

Project files. A project is a folder with a `project.json`, one subfolder per tool (`wing/`, `cad/`, `flow/`, `lapsim/`) and `jobs/<timestamp-id>/` for every run. Plain JSON everywhere so it is easy to inspect and diff.

Dependencies

Where	What
Python	numpy, scipy, pandas, gmsh (pip wheel), optuna, OCP via cadquery-ocp (STEP and IGES tessellation), trimesh, opencv-python, scikit-image, pytest
External	SU2 for 2D RANS. Find the cleanest working install on Apple silicon (conda-forge if it has an osx-arm64 build, otherwise build from source with meson and ninja) and write the exact steps in README.md
Swift	No third-party packages unless something genuinely needs one. Note any in DECISIONS.md

Units. SI internally everywhere. CAD and geometry shown in mm, angles shown in degrees, stored in radians.

4 Phase 0. Foundations

- Xcode project builds and launches, sidebar shows the four tools as empty screens.
- uv environment for the engine, plus `aeroengine ping` that returns a JSON result.
- Job runner in Swift. Start, stream progress, cancel, view log, and show errors as messages rather than crashes. A jobs panel lists running and finished jobs.
- Create, open and save projects. Recent projects list.
- Settings for engine path, SU2 path, max parallel jobs and cores per job.
- Housekeeping. Each job deletes intermediates it does not need when it finishes (meshes kept only if the user ticks keep mesh). Nothing gets written outside the project folder and the app's own cache.

Acceptance checks

☐ Ping round trip works from the UI.

☐ A 10 second dummy job shows live progress and cancels cleanly.

☐ Killing the engine process mid-job shows an error and the app keeps running.

5 Phase 1. Lap Sim

First because it is pure Python, quick to get right, and proves the Swift to Python pipeline and Swift Charts before the hard tools.

Model

Quasi-steady point mass. The track centreline (x, y points) is resampled to about 0.5 m spacing and curvature κ comes from a smoothed spline. At each point the grip-limited cornering speed includes downforce.

```
m v² k = mu_lat (m g + ½ rho ClA v²)

v² = mu_lat m g / (m k - ½ mu_lat rho ClA)
(if the denominator is <= 0, grip never limits, cap at v_max)
```

A forward pass handles acceleration. Drive force is the smaller of power over speed and traction (μ_long times the normal load on the driven axle, including its share of downforce), minus aero drag ½ρCdA v² and rolling resistance, reduced by a friction circle using the lateral grip already in use. A backward pass does the same for braking with all four tyres. Take the minimum speed of the passes at each point and integrate time.

Inputs

Mass with driver, ClA, CdA, μ_lat, μ_long, peak power, driven axle load fraction, rolling resistance coefficient, air density, v_max. All editable and saved in the project. Ship placeholder values, clearly labelled as placeholders, since Dominic will fill in the team's real numbers.

Tracks

Import a centreline as CSV. Built-in skidpad (centreline radius 9.125 m, from the 15.25 m inner circle plus a 3 m lane) and a 75 m acceleration run. Also include one synthetic test track. Real event layouts come from Dominic.

Outputs

- Lap time, plus total time and an energy estimate (integral of drive force times speed) for an endurance lap count.
- Speed vs distance plot and a track map coloured by speed.
- Config A vs config B comparison with a time delta trace.
- Sensitivities ∂t/∂ClA and ∂t/∂CdA by finite differences, and the exchange rate between them, meaning how much ClA is worth one unit of CdA at equal lap time. Wing Lab will use this number later.

Acceptance checks

☐ Skidpad with ClA = 0 gives v = sqrt(μ_lat g R) exactly.

☐ Acceleration run matches a hand calculation within 1%.

☐ Adding ClA never makes a lap slower. Adding CdA never makes it faster.

☐ Lap time changes by under 0.5% when the resampling step is halved.

6 Phase 2. CAD Viewer and the shared 3D core

Build the 3D core (camera, scene, mesh loading, picking, colour mapping) as its own module, not inside the CAD tool, because the Flow Field Viewer reuses all of it.

- STEP and IGES go through the engine. OCP tessellates them with coarse, medium and fine presets and writes a .glb that keeps part names. STL and OBJ load directly in Swift.
- Drag and drop a file onto the window to open it. Cache tessellations so reopening is instant.
- Orbit, pan and zoom with trackpad gestures. Fit all, standard views (front, side, top, iso) and a perspective or orthographic toggle.
- Part tree from the assembly structure with visibility toggles.
- Click two surface points to measure distance in mm. Bounding box dimensions readout.

Acceptance checks

☐ Opens a STEP exported from CATIA (Dominic supplies a sample).

☐ A model of about 1 million triangles orbits smoothly.

☐ Measuring a known dimension matches within 0.1 mm.

7 Phase 3. Flow Field Viewer

Input is a video (mp4 or mov) or an image sequence of Cp0 section planes stepping front to back through the flow, plus a capture config. Accuracy is fine for visual exploration, not final numbers.

Calibration wizard

1. Draw the crop rectangle around the plot region on one frame.
2. Pick the colormap from presets (STAR-CCM+ defaults, viridis, jet, rainbow) or sample it by drawing a line along a legend in a frame.
3. Enter the value range of the colormap.
4. Set the spatial scale by clicking two points a known distance apart, or by entering the plane width and height.
5. Set plane positions, either a start and end position with even spacing or a CSV of one position per frame, plus a frame stride.
6. Pick mask colours (geometry, background) that become empty space.
7. Save the whole calibration as a reusable JSON config, so the same capture setup works on any later video.

Reconstruction

Build a lookup table of the colormap in CIELAB. Map each pixel to the nearest colour, and mark it empty if the colour distance is above a threshold. Optional median filter. Downsample each plane to a working resolution (around 256 px wide by default) for speed. Stack the planes into a 3D volume and save it as .npz with its metadata.

Display

- Isosurface at a chosen Cp0 value via marching cubes, with a slider. Several isosurfaces at once in different colours.
- Value window mode that shows only voxels between two chosen values.
- Slice scrubber that shows the original video frame next to the reconstructed plane.
- Axes in real units. Export isosurfaces as .obj or .glb and the volume as .npz or .vti.
- A short capture guide inside the tool. Lossless or high bitrate video, fixed colour range, fixed camera, constant plane step, legend hidden unless it is used for sampling.

Acceptance checks

☐ Synthetic regression test. Generate frames from a known analytic 3D field with a known colormap, encode them as H.264, then reconstruct. Recovered values within 3% of the colour range and isosurface volume within 5% of the truth.

☐ A real STAR-CCM+ Cp0 video from Dominic reconstructs into a recognisable wake.

8 Phase 4. Wing Lab

The biggest tool. Build it in this order. Library, then canvas, then inviscid preview, then one SU2 run end to end, then the optimisation loop.

8.1 Airfoil library

A built-in database of airfoils that Dominic can browse and drag straight onto the canvas.

- Source is the UIUC Airfoil Coordinates Database (around 1,600 sections) plus a NACA 4 and 5 digit generator. A script, `engine/scripts/fetch_airfoils.py`, downloads it into `data/airfoils/`. Check UIUC's terms. If redistribution is unclear, keep the script and do not commit the files themselves.
- Normalise every section to Selig format, unit chord, leading edge at the origin, re-panelled with cosine spacing to a consistent point count. Record trailing edge thickness rather than silently closing it.
- Index (SQLite) with name, family, max thickness and its position, max camber and its position, trailing edge thickness and source.
- Import your own .dat files (Selig or Lednicer, detected automatically) into a My Airfoils section.
- Library panel with search, filters for thickness, camber and family, a small profile thumbnail for every entry, and favourites.
- Later, not v1. Precomputed polars at FS Reynolds numbers for each section.

8.2 Canvas

- Drag an airfoil from the library onto the 2D canvas to add an element. Elements are ordered main, flap 1, flap 2 and so on.
- Drop an airfoil onto an existing element to swap its section while keeping its chord, angle and position, so trying a different profile takes one drag.

- Each element can be dragged to move it, rotated with a handle to set its angle, and resized with a handle to set its chord. Inverted for downforce by default, with a flip toggle.
- Live readouts per element. Chord, angle, and the gap and overlap to the element in front. Gap is the minimum distance from the previous element's trailing edge to this element's surface. Overlap is measured along the main element's chord line. All of these are editable as numbers too, so typing a gap moves the element.
- Optional snapping and per-value locks. Undo and redo.
- Rules box overlay. Dominic enters the bounding box limits from the current FS rules. Never hardcode rule values.
- Collision check. Elements that intersect or sit closer than a minimum gap get flagged.

8.3 Inviscid preview

A multi-element linear vortex panel method recomputes Cl per element and the Cp distribution as you drag, fast enough to feel live (under about 50 ms). Label it clearly as inviscid preview. It ignores the viscous slot effects that actually set the best gap, so it is for angle trends and sanity checks only.

8.4 Single 2D RANS run

- Meshing in gmsh. Boundary layer prisms on every element with first cell height set for y+ around 1 at the run's Reynolds number, growth rate around 1.15 to 1.2, enough layers to contain the boundary layer. Triangles elsewhere, a refinement region along each wake, far field circle around 50 chords out. Freestream only in v1 (rear wing), ground plane later.
- Solver is SU2 incompressible RANS, Spalart-Allmaras by default with an SST option, steady.
- Inputs are speed, density, viscosity and reference chord. The app shows the resulting Reynolds number.
- Outputs are Cl, Cd and Cm per element and in total, convergence history, surface Cp per element, and a field plot.
- Converged means residuals dropped by a set amount and Cl and Cd have stopped changing over a window. Runs that fail this are flagged, never shown as good results.
- Target runtime is a few minutes per run using about 4 cores.

8.5 Optimisation

- In the canvas, tick which values are design variables (angle, gap and overlap per element) and give each one bounds.
- Objectives. Maximum Cl, maximum L/D, maximum Cl with a Cd cap, minimum Cd with a Cl target. A lap time weighted objective using the Lap Sim exchange rate comes later.
- Optuna (TPE by default, CMA-ES as an option) with parallel trials, for example 4 trials at 4 cores each, set in Settings.
- Constraints are geometric validity, minimum gap and the rules box. Trials with failed meshes or diverged runs are recorded as failed and the study carries on.
- Results view with a sortable table of trials, a best-so-far plot, a Cl vs Cd scatter with the Pareto front, and click any trial to load it into the canvas.
- Studies are stored in the project (Optuna SQLite storage) so an interrupted study can be resumed.

8.6 Shape optimisation (later phase, design for it now)

CST parametrisation per element (around 6 upper and 6 lower coefficients), SU2 discrete adjoint with an FFD box for gradients, alternating with the gap and angle search. Constraints for minimum thickness ratio, minimum trailing edge thickness, a spar box the section must contain, and a curvature limit to stop wiggly shapes. Not built in v1, but store elements as coordinate arrays plus an optional parametrisation so it slots in cleanly.

8.7 Export

Element coordinates as .dat files and as CSV point sets in mm, placed in the wing's reference frame so they can go straight into CATIA.

Acceptance checks

- ☐ Single element. NACA 0012 at Re 6 million against the Ladson experimental data, lift slope and drag in the expected range.
- ☐ Multi-element. A public multi-element case such as the 30P30N high-lift configuration, Cl vs angle within a few percent before stall. If its geometry is not freely obtainable, use another public case and note it in DECISIONS.md.
- ☐ The inviscid preview converges as panel count increases and matches a high panel count reference.
- ☐ A mesh independence check for the default mesh settings, written up in DECISIONS.md.
- ☐ An optimisation of two variables on a 2-element wing finishes overnight without manual intervention.

9 Phase plan

Phase	Delivers	Done when
0 Foundations	App shell, engine, job runner, projects, settings	Section 4 checks pass
1 Lap Sim	Working lap sim with plots and sensitivities	Section 5 checks pass
2 CAD Viewer	Shared 3D core plus CAD tool	Section 6 checks pass
3 Flow Field Viewer	Video to 3D Cp0 field	Section 7 checks pass
4 Wing Lab	Library, canvas, preview, SU2 run, optimiser	Section 8 checks pass
5 Later	Shape optimisation, sage37 visual polish, packaging	Planned after v1 is in use

After each phase, stop and show Dominic the result before starting the next one.

10 Later, not v1

- Visual polish in Dominic's sage37 design system, built from the `sage37-swift` skill. It holds every colour, type style, glass tier, component and the Metal refraction shader, so follow it rather than inventing a look. Machine output (SU2 residuals, optimiser logs, the job log) goes in one glassout console per screen, built from the `glassout-swift` skill.
- STAR-CCM+ as an alternative Wing Lab backend, driven by Java macros in the Parallels VM.
- Overlaying CAD geometry inside the Flow Field Viewer.
- Lap time weighted objective in Wing Lab using the Lap Sim exchange rate.
- Polar database for the airfoil library, ground effect option, run reports as PDF.
- Packaging the engine inside the .app, signing and sharing with the team.

11 Working rules for Claude Code

- Git from the first commit. Commit after every step that works.
- Every numeric module gets pytest tests. Run them all before calling a phase done.
- Keep README.md (setup that actually works) and docs/DECISIONS.md current.
- Never fake or smooth over results. A failed mesh or diverged run is shown as failed.
- Clean up after yourself. No stray temp files, no files outside the repo or project folders.
- Keep v1 on default SwiftUI controls. When styling starts, load `sage37-swift` first and use its tokens. Never hardcode colours or sizes outside it.
- Dominic's call, ask him for these. Team car parameters, current FS rules box limits, a sample STEP from CATIA, a sample Cp0 video with its STAR colormap and range, real track layouts, and any change of scope.

12 First prompt

Put this PDF at `docs/AeroBench_spec.pdf` in an empty folder, open Claude Code there and start with this.

```
Read docs/AeroBench_spec.pdf in full. Set up the repo as described
in section 3, then complete Phase 0. When the app launches and the
ping job round trip works, show me and wait before starting Phase 1.
```